Data Analytics Capstone

Predicting Service Level Agreement Breach in IT Incident Management: An Analysis of Ticket-Level Drivers in an Enterprise ServiceNow Event Log

Interim Report

Binesh Balakrishnan

Walsh College

QM640: Data Analytics Capstone

Ms Keya Choudhury Ganguli

Term 3

October 27, 2026

GitHub Repository (Data and Code)

All data files and code used in this report are available in the public GitHub repository:

https://github.com/binesh29/QM640_data_analytics_capstone

The repository contains the unchanged source data (data/raw), the incident-level analysis file (data/processed), four Jupyter notebooks that reproduce every table and figure in this report (notebooks/01 to 04), the numeric outputs in JSON format (results), the figures (figures), and the sample size script (src/sample_size.py). The README gives the installation and run instructions; Appendix B lists which notebook produces each table and figure.

Table of Contents

Introduction....4
Background and Context....4
Problem Statement....5
Purpose of the Study....5
Interim Project Status (Progress Snapshot)....5
Scope and Objectives....6
Research Questions and Hypotheses....7
Literature Survey....8
Literature Review Approach....8
Summary of Key Literature....9
Data Description....12
Data Source(s) and Access....12
Dataset Overview....12
Data Dictionary....13
GitHub Data Availability Statement....16
Screenshots / Evidence....17
Analysis....18
Data Cleaning....18
EDA Results....21
Sample Size Computation....28
Modelling....29
Choice of Models with Justification....30
Features Included and Feature Engineering....31
Evaluation Metrics....33
Preliminary Results....34
Preliminary Findings by Research Question....35
Preliminary Model Performance....36
Interim Limitations and Risks....40
Next Steps (for Final Report)....41
References....42
Appendix A....47
Appendix B....48

Introduction

Background and Context

Incident management is the IT service management (ITSM) process that restores normal service as quickly as possible after an unplanned interruption. In enterprise environments it runs on a ticketing platform such as ServiceNow, where each incident is logged, prioritised from its impact and urgency, routed to a support group, worked, resolved, and closed. Frameworks such as the IT Infrastructure Library (ITIL) formalise this lifecycle, and organisations that adopt them report gains in service quality alongside continuing difficulty in measuring and improving process performance (Marrone & Kolbe, 2011).

In managed services, the performance of this process is governed by a service level agreement (SLA): a contractual commitment to resolve incidents within target times that depend on priority. An SLA breach is therefore a commercial event as well as an operational one. Breaches erode customer confidence, can trigger service credits in outsourcing contracts, and consume the time of the service delivery managers who must explain them in monthly reviews. The stakeholders are the end users who experience the outage, the service desk and resolver groups who work the tickets, the service delivery manager accountable for SLA attainment, and the client who pays for the service.

Two operational behaviours are widely seen as drivers of delay: reassignment, where a ticket is passed between support groups before it reaches one that can resolve it (Shao et al., 2008), and the use or non-use of recorded knowledge. At the same time, artificial intelligence for IT operations (AIOps) has created interest in using historical operational data to predict and prevent service problems rather than react to them (Dang et al., 2019). The research basis for

such prediction is predictive process monitoring, which uses event logs to forecast the outcome or remaining time of running cases (van der Aalst, 2016; Teinemaa et al., 2019).

Problem Statement

In enterprise incident management, SLA attainment is monitored after the fact: breaches are counted at month end, and the ticket attributes and process behaviours that preceded them are rarely tested statistically. Service delivery managers therefore make routing and knowledge-management decisions on intuition, and at-risk tickets are escalated only once the SLA clock is nearly exhausted. The problem this study addresses is the absence of evidence on which ticket-level factors are significantly associated with SLA breach, and whether breach risk can be predicted at the moment a ticket is logged more accurately than by priority alone. The study has one dependent variable, SLA breach (binary: 1 = breached, 0 = met).

Purpose of the Study

The study aims to identify the ticket-level drivers of SLA breach in an enterprise incident-management process and to test whether a model built only on attributes available at logging can predict breach risk, so that operations can escalate risky tickets earlier and route them better. It is both explanatory, through hypothesis tests on drivers (RQ1 to RQ3), and predictive, through binary classification (RQ4). This interim report documents the data, the cleaning and exploratory analysis, the sample size, the modelling approach, and the preliminary results for all four research questions.

Interim Project Status (Progress Snapshot)

- **Completed:** data download and validation (141,712 events, 36 columns); collapse to 24,918 incidents; cleaning log; exploratory data analysis with six figures; sample size confirmation; first run of the χ^2 tests and logistic regressions for RQ1 to RQ3; first run of

the baseline and three machine-learning models for RQ4 with the DeLong test; all code published in the GitHub repository.

- **In progress:** sensitivity analyses for a structural break found in March 2016 (see EDA Results); time-aware cross-validation for RQ4; probability calibration and choice of an operating threshold.
- **Pending:** final model selection and tuning, interpretation of feature effects, translation of results into an escalation rule, final report and presentation.

Scope and Objectives

The study uses one publicly available event log extracted from the ServiceNow audit system of a real IT company (Amaral et al., 2018b). The unit of analysis is the incident: the 141,712 events are collapsed to 24,918 incidents so that each incident contributes one observation. The objectives are measurable and each maps to one analysis: (a) quantify how the SLA breach rate differs across the four priority levels (χ^2 test, Cramér's V); (b) estimate the odds ratio of breach per additional reassignment after controlling for priority and category (logistic regression); (c) estimate the odds ratio of breach for incidents where a knowledge-base article was used, with the same controls (logistic regression and Mantel–Haenszel odds ratio); and (d) build a breach-prediction model from at-logging attributes and test whether its area under the ROC curve (AUC) is significantly higher than that of a priority-only model (DeLong test).

Boundaries. In scope are the four research questions, descriptive profiling, and a validated prototype model, all on this dataset. Out of scope are the design of SLA targets (which are not recorded), staffing optimisation, causal claims about interventions, real-time deployment, and text analytics (the log has no free-text fields). Resolution time is used only to describe the data. **Feasibility.** The data is free and already downloaded, every incident has a recorded

outcome, the full analysis runs on a laptop with open-source Python libraries in under ten minutes, and the 10-week schedule leaves five weeks after this report for refinement. Table 1 lists the variables used.

Table 1

Study Variables: Role, Type, and Measurement

Variable	Role	Type	Measurement and units	Used in
sla_breach	**Dependent variable**	Binary	1 = breached, 0 = met (NOT made_sla, final record)	**RQ1–RQ4**
priority	Independent variable; control	Ordinal, 4 levels	1 Critical, 2 High, 3 Moderate, 4 Low	RQ1; control in RQ2–RQ3
reassignment_count / reassigned	Independent variable	Count / binary	Number of reassignments, 0–27; reassigned = 1 if at least one	RQ2
knowledge	Independent variable	Binary	1 = knowledge-base article used	RQ3
category	Control variable	Nominal	Service category; levels with fewer than 30 incidents grouped as Other	RQ2, RQ3
At-logging attributes (log_ prefix)	Predictors	Mixed	First event record of each incident (see Table 8)	RQ4

Note. Final-record values are used for RQ1 to RQ3; first-record (at-logging) values are used for RQ4.

Research Questions and Hypotheses

Each research question is tested at $\alpha = 0.05$ and shares the same dependent variable, SLA breach. In the notation below, H0 and H1 denote the null and alternative hypotheses and the subscript gives the research question.

Research Question 1 (RQ1)

Does the SLA breach rate differ significantly across the four priority levels?

$H0_1$: SLA breach is independent of priority. $H1_1$: SLA breach is associated with priority.

Research Question 2 (RQ2)

Is the number of reassignments associated with the likelihood of SLA breach, after controlling for priority and service category?

$H0_2$: odds ratio for reassignment count = 1 ($\beta_1 = 0$). $H1_2$: odds ratio ≠ 1 ($\beta_1 \neq 0$).

Research Question 3 (RQ3)

Is knowledge-base use associated with the likelihood of SLA breach, after controlling for priority and service category?

$H0_3$: odds ratio for knowledge use = 1 ($\beta_1 = 0$). $H1_3$: odds ratio ≠ 1 ($\beta_1 \neq 0$).

Research Question 4 (RQ4)

Can a machine-learning classifier using only attributes available at ticket logging predict SLA breach with a significantly higher AUC than a priority-only baseline model?

$H0_4$: AUC of the best model = AUC of the priority-only baseline. $H1_4$: AUC of the best model > AUC of the baseline.

Literature Survey

Literature Review Approach

Sources were identified through keyword searches of Google Scholar, Crossref, and the ACM, IEEE, and Springer digital libraries, using the keywords "incident management", "ticket routing", "SLA violation prediction", "predictive process monitoring", "outcome prediction event log", and "data leakage". Peer-reviewed journal articles, conference papers, and standard methodological texts were included when they addressed IT service operations, prediction from event logs, or the statistical and machine-learning methods used here. All bibliographic details were checked against Crossref. Table 2 summarises the most relevant sources and links each to the research questions.

Summary of Key Literature

Table 2

Literature Relevance Matrix

Author (Year)	Domain / context	Dataset / setting	Method(s)	Key findings	How it supports the RQs
Marrone & Kolbe (2011)	ITSM frameworks	Survey of organisations adopting ITIL	Empirical survey analysis	Benefits grow with adoption, but measuring process performance remains hard	Motivates measuring SLA drivers (all RQs)
Shao et al. (2008)	Ticket routing	Historical problem-ticket transfer sequences	Resolution-sequence mining (Markov model)	Mined routing reduces the transfers needed to reach a resolver group	Reassignment as a cost driver (RQ2)
Botezatu et al. (2015)	Ticket dispatching	Enterprise incident tickets	Multi-view clustering	Clustering ticket views supports dispatching to the right group	First-time-right routing (RQ2, RQ4)
Kikuchi (2015)	Incident management	Incident ticket update histories	Prediction from update history	Ticket update history can be used to predict incident workload	Uses ticket logs for prediction (RQ4)
Leitner et al. (2010)	Service-oriented computing	Composite web services	Runtime monitoring and machine-learning prediction	SLA violations can be predicted during execution and prevented	SLA breach as a prediction target (RQ4)
Amaral et al. (2018a, 2019)	Process mining	This incident log (UCI)	Filter and wrapper attribute selection; completion-time prediction	Selected attributes improve completion-time prediction	Same data; a gap for breach classification (RQ4)
Teinemaa et al. (2019)	Predictive process monitoring	24 tasks from 9 real event logs	Benchmark of 11 outcome-prediction methods	Taxonomy and benchmark of outcome-oriented methods	Outcome (breach) prediction design (RQ4)
Verenich et al. (2019); Tax et al. (2017)	Predictive process monitoring	Public event logs	Remaining-time benchmarks; LSTM networks	Remaining time and next events can be predicted from running cases	Contrast: in-flight vs at-logging prediction (RQ4)
Kaufman et al. (2012)	Data mining practice	Competitions and real projects	Formal definition of leakage	Leakage inflates performance; avoid with learn–predict separation	Feature exclusion rules (RQ4)

Author (Year)	Domain / context	Dataset / setting	Method(s)	Key findings	How it supports the RQs
Grinsztajn et al. (2022)	Machine learning on tabular data	Benchmark of tabular datasets	Tree ensembles vs deep learning	Tree-based models remain strong on typical tabular data	Choice of RF and XGBoost (RQ4)
Saito & Rehmsmeier (2015)	Classifier evaluation	Simulated and real imbalanced data	ROC vs precision–recall analysis	Precision–recall plots are more informative under imbalance	Report PR-AUC with AUC (RQ4)
Mantel & Haenszel (1959); Hosmer et al. (2013)	Statistical methods	Stratified and observational data	Pooled odds ratio; logistic regression	Adjusted odds ratios separate a factor from confounders	Adjusted tests (RQ2, RQ3)

Note. Key findings are paraphrased from the sources' abstracts and stated aims.

Process performance and routing in ITSM. Studies of ITSM adoption show that organisations gain service quality from structured processes yet struggle to measure what drives performance (Marrone & Kolbe, 2011). Research on ticket routing treats reassignment as waste: Shao et al. (2008) mine historical transfer sequences to recommend the next resolver group, and Botezatu et al. (2015) cluster tickets on several views of their content to dispatch them correctly the first time. Both works optimise routing but measure success by transfers or dispatch accuracy, not by contractual SLA outcomes, and both rely on text or sequence data that are often unavailable. Kikuchi (2015) shows that ticket update histories carry predictive signal for workload. Together these studies support RQ2, but none estimates how much each reassignment changes the odds of breaching an SLA after accounting for priority.

Predicting outcomes from event logs. Predictive process monitoring forecasts the outcome or remaining time of running cases from event logs (van der Aalst, 2016). Benchmarks show that no single technique dominates: Teinemaa et al. (2019) compare 11 outcome-oriented methods on 24 tasks, and Verenich et al. (2019) compare remaining-time methods, including the long short-term memory (LSTM) networks of Tax et al. (2017) and the approach of Polato et al. (2018), which predicts both the remaining time and the sequence of future activities. These

methods predict while a case is running, using the events observed so far. Leitner et al. (2010) predict SLA violations in composite web services from runtime monitoring data. The incident log used here has been studied only for completion-time prediction with attribute selection (Amaral et al., 2018a, 2019). The strength of this literature is its rigorous benchmarking; its limitation for service desks is that in-flight predictions arrive after reassignments and updates have already happened, which is too late for an escalation decision at logging.

Model choice and validity for tabular operational data. Incident attributes are mostly categorical with many levels, such as assignment group, location, and symptom. Tree ensembles, namely random forests (Breiman, 2001) and gradient boosting (Chen & Guestrin, 2016), remain among the strongest models for such tabular data (Grinsztajn et al., 2022), while regularised logistic regression offers interpretability (Hosmer et al., 2013). High-cardinality fields need encoding choices such as grouping rare levels or target encoding (Micci-Barreca, 2001). The most important validity threat is leakage, using information that would not be available at prediction time (Kaufman et al., 2012), which in this log includes reassignment and update counts. AIOps practitioners also stress data quality and shifting operating conditions (Dang et al., 2019), which supports a time-ordered train–test split rather than a random one.

Evaluation and inference. For imbalanced outcomes, AUC summarises ranking quality across thresholds (Fawcett, 2006), but precision–recall measures are more informative about the minority class (Saito & Rehmsmeier, 2015), and the Brier score assesses probability accuracy (Brier, 1950). Two AUCs computed on the same test set are compared with the DeLong et al. (1988) test. For the explanatory questions, Mantel–Haenszel pooling (Mantel & Haenszel, 1959) and multivariable logistic regression estimate associations adjusted for confounders, and the Holm (1979) procedure controls the family-wise error of pairwise tests.

Gap and justification of the approach. The literature optimises routing, predicts process outcomes during execution, and predicts SLA violations in web services, but no study found tests the managerial hypotheses that service delivery managers act on, namely priority, reassignment, and knowledge use, against SLA breach in an ITSM log, or frames breach as a classification problem at the moment of logging against the priority rule practitioners already use. This study fills that gap by combining adjusted hypothesis tests (RQ1 to RQ3) with a leakage-free, time-ordered comparison of a priority baseline, regularised logistic regression, random forest, and XGBoost (RQ4).

Data Description

The data is a real operational event log from an IT company's ServiceNow instance, enriched with fields from the platform's database and anonymised by its donors. It is not synthetic and is not sourced from Kaggle.

Data Source(s) and Access

Amaral et al. (2018b) donated the Incident Management Process Enriched Event Log to the University of California, Irvine (UCI) Machine Learning Repository, at https://archive.ics.uci.edu/dataset/498/incident+management+process+enriched+event+log (https://doi.org/10.24432/C57S4H). It is released under a Creative Commons Attribution 4.0 International (CC BY 4.0) licence, which permits redistribution with attribution, so the unchanged file is also stored in the project's GitHub repository.

Dataset Overview

- **Number of records:** 141,712 events (rows), which collapse to 24,918 incidents.
- **Number of variables:** 36 source columns, plus derived variables listed in Table 3.

- **Time period:** incidents opened from 29 February 2016 to 16 February 2017; 98.1% were opened between March and May 2016.
- **Unit of analysis:** the incident (one row per incident after collapsing the event log).
- **Target variable:** sla_breach = 1 if the incident did not meet its SLA (made_sla = false on the final record), else 0. Prevalence is 36.6% (9,115 incidents).

Each event row is a snapshot of an incident after an update, so one incident appears on 5.7 rows on average (Figure 1, in Screenshots / Evidence below). Analysing rows directly would count incidents many times and overstate significance, so each incident is represented by its first record, which holds the attributes known at logging and is used for RQ4, and its final record, which holds the outcome and accumulated counts and is used for RQ1 to RQ3. The UCI description of made_sla is ambiguous; the data confirms that true means the SLA was met, because median resolution time (excluding 39 zero-duration incidents) is 1.1 hours when true and 201.7 hours when false.

Completeness and limitations. The outcome is recorded for every incident. Five columns are 98% to 100% empty (caused_by, vendor, cmdb_ci, rfc, problem_id) and are not used, and symptom is missing for 23.0% of incidents at logging. The data comes from one anonymised organisation, category and group names are coded (for example, Category 26), SLA targets are not recorded, and almost all incidents fall in a three-month window. These limit generalisation and are discussed with the results. Analyst identifiers are not used, so the work rates tickets rather than people.

Data Dictionary

Table 3 defines every source column and each derived variable, with its data type, range, and the treatment of missing values.

Table 3

Data Dictionary

Variable name	Definition	Data type	Allowed values / range	Missing values handling	Notes
number	Incident identifier	String (ID)	INC codes; 24,918 unique	None missing	Unit-of-analysis key
incident_state	State of the incident at that event	Categorical	8 states + "-100"	None missing; 5 "-100" rows kept	Final record ends in Closed
active	Whether the record is active	Boolean	true / false	None missing	Data check only
reassignment_count	Times the group or analyst changed	Integer	0–27	None missing	RQ2 IV; excluded from RQ4
reopen_count	Times the caller rejected the resolution	Integer	0–8	None missing	Descriptive; excluded from RQ4
sys_mod_count	Number of updates to that event	Integer	0–129	None missing	Event ordering; excluded from RQ4
made_sla	Whether the SLA target was met	Boolean	true = met	None missing	Source of the DV
caller_id	Affected user	String (ID)	5,244 labels	0%; "Missing" level	RQ4 as training-period frequency
opened_by	User who reported the incident	String (ID)	Labels	3.4%; not used	Not used
opened_at	Incident opening time	Datetime	29 Feb 2016 – 16 Feb 2017	None missing	Hour/day features; split order
sys_created_by	User who registered the incident	String (ID)	Labels	37.5%; not used	Not used
sys_created_at	System creation time	Datetime	Timestamps	37.5%; not used	Not used
sys_updated_by	User who made the update	String (ID)	Labels	None missing; not used	Not used
sys_updated_at	Time of the update	Datetime	Timestamps	None missing	Tie-break for event order
contact_type	Reporting channel	Categorical	5 levels (99.1% phone)	None missing	RQ4 predictor

Variable name	Definition	Data type	Allowed values / range	Missing values handling	Notes
location	Affected location	Categorical	224 labels	0.1%; "Missing" level	RQ4 predictor
category	First-level affected service	Categorical	52 levels (incident level)	0.1%; "Missing" level	Control (RQ2–RQ3); RQ4
subcategory	Second-level affected service	Categorical	254 labels	0.1%; "Missing" level	RQ4 predictor
u_symptom	User-reported symptom	Categorical	525 labels	23.3%; "Missing" level	RQ4 predictor
cmdb_ci	Affected configuration item	String (ID)	Labels	99.7%; missing flag only	RQ4 (flag)
impact	Impact of the incident	Ordinal	1 High, 2 Medium, 3 Low	None missing	RQ4 predictor
urgency	Urgency stated by the user	Ordinal	1 High, 2 Medium, 3 Low	None missing	RQ4 predictor
priority	Calculated from impact and urgency	Ordinal	1 Critical – 4 Low	None missing	RQ1 IV; control; RQ4 baseline
assignment_group	Support group in charge	Categorical	78 labels	10%; "Missing" level	RQ4 predictor (at logging)
assigned_to	Analyst in charge	String (ID)	Labels	19.4%; not used	Not used (privacy)
knowledge	Whether a knowledge-base article was used	Boolean	true / false	None missing	RQ3 IV; RQ4 sensitivity
u_priority_confirmation	Whether priority was double-checked	Boolean	true / false	None missing	RQ4 predictor
notify	Whether notifications were generated	Categorical	2 levels	None missing	RQ4 predictor
problem_id	Associated problem record	String (ID)	Labels	98.4%; not used	Too sparse
rfc	Associated change request	String (ID)	Labels	99.3%; not used	Too sparse
vendor	Vendor in charge	String (ID)	Labels	99.8%; not used	Too sparse

Variable name	Definition	Data type	Allowed values / range	Missing values handling	Notes
caused_by	Change that caused the incident	String (ID)	Labels	100%; not used	Empty
closed_code	Resolution code	Categorical	17 levels	0.5%; not used	Known only at closure
resolved_by	User who resolved the incident	String (ID)	Labels	0.2%; not used	Not used (privacy)
resolved_at	Time of resolution	Datetime	Timestamps	2.2% of events; 1,556 incidents left missing	Descriptive only
closed_at	Time of closure	Datetime	Timestamps	None missing	Descriptive only
sla_breach (derived)	1 if made_sla is false on the final record	Binary	0 / 1	None missing	**DV, RQ1–RQ4**
reassigned (derived)	1 if reassignment_count ≥ 1	Binary	0 / 1	None missing	RQ2 χ^2 test
resolution_hours (derived)	resolved_at − opened_at	Continuous	0 – 8,070 hours	1,556 missing; left missing	Descriptive only
log_hour, log_dayofweek (derived)	Hour and weekday of opened_at	Integer	0–23; 0 (Mon) – 6 (Sun)	None missing	RQ4 (cyclical hour, weekend flag)
caller_freq (derived)	Training-period incident count of the caller, log(1 + n)	Continuous	≥ 0	Unseen callers = 0	RQ4 predictor

Note. Missing percentages are at event level unless stated. IV = independent variable; DV = dependent variable. Adapted from Amaral et al. (2018b).

GitHub Data Availability Statement

The raw and processed datasets and all code are available in the GitHub repository https://github.com/binesh29/QM640_data_analytics_capstone, organised as shown in Figure 2: raw data in data/raw/incident_event_log.csv, the processed incident-level file in data/processed/incidents.csv, and code in notebooks/ (01 to 04) and src/.

Screenshots / Evidence

Figure 1 shows a preview of the raw event log, and Figure 2 shows the folder structure of the repository.

Figure 1

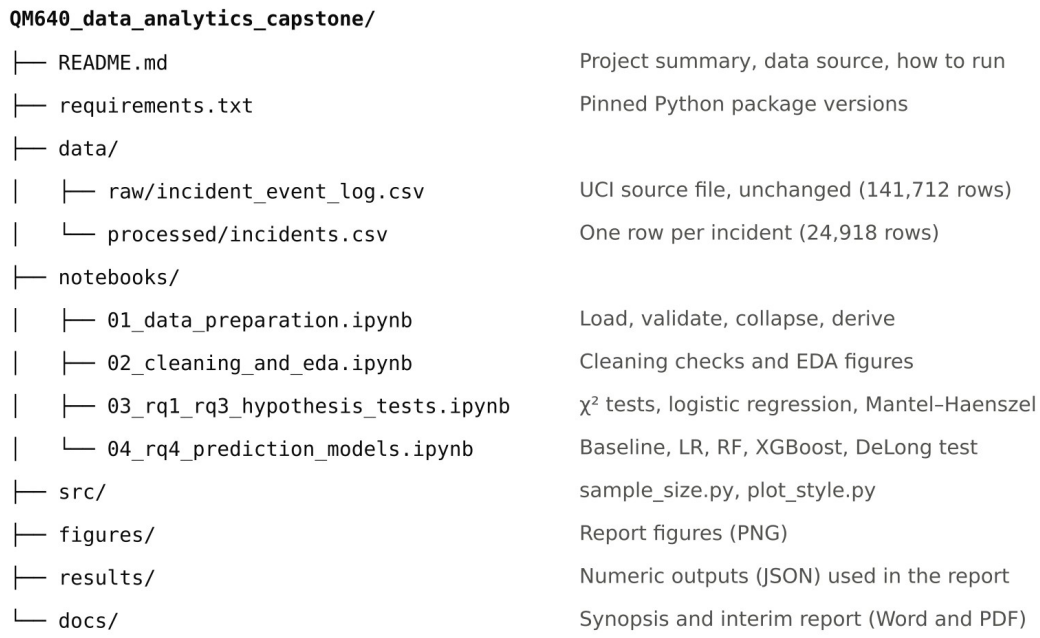
Extract of the Raw Event Log (First Six Events, Eight of 36 Columns)

number	incident_state	reassignment_count	made_sla	opened_at	priority	knowledge	resolved_at
INC0000045	New	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Resolved	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Resolved	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Closed	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000047	New	0	true	29/2/2016 04:40	3 - Moderate	true	1/3/2016 09:52
INC0000047	Active	1	true	29/2/2016 04:40	3 - Moderate	true	1/3/2016 09:52

Note. Each incident (number) appears on several rows as it moves through states. From *Incident Management Process Enriched Event Log* [Data set], by C. Amaral, M. Fantinato, and S. Peres, 2018, UCI Machine Learning Repository (https://doi.org/10.24432/C57S4H). CC BY 4.0.

Figure 2

Structure of the GitHub Repository



Note. Screenshot of the folder structure of the public repository, https://github.com/binesh29/QM640_data_analytics_capstone.

Analysis

The analysis has three parts: data cleaning, exploratory data analysis (EDA) linked to each research question, and confirmation of the sample size. Notebook 02 produces every number and figure in this section.

Data Cleaning

Cleaning was applied conservatively: every source row is kept, and treatments are recorded in Table 4 with counts. The missing-value code "?" was converted to missing, and timestamps were parsed as day-first dates. No exact duplicate rows exist, but 5 pairs of events share the same incident number and update counter; these ties are broken by update time so that the first and final records are well defined, and the collapsed file has no duplicate incidents. Five

rows carry an undocumented state code of -100; they belong to two incidents whose final record is Closed, so they do not affect the outcome.

Missing categorical values at logging were kept as an explicit "Missing" level rather than imputed, because missingness itself may carry information; for example, an incident logged without an assignment group (3.6%) has not yet been routed. Outliers were detected with the interquartile range (IQR) rule. They are genuine operational values rather than errors, so they were retained: 2,848 incidents have more than two reassignments (maximum 27), and 812 took more than 1,000 hours to resolve. Removing them would delete exactly the difficult incidents the study is about. Instead, reassignment is also analysed in bands (0, 1, 2, 3+) to check linearity, and resolution time is shown on a log scale and used only descriptively.

Table 4

Data Cleaning Log

Issue	Variables affected	Detection method	Treatment applied	Rationale
Missing values (sparse)	caused_by, vendor, cmdb_ci, rfc, problem_id	Share of "?" per column: 98.4%–100%	Not used; cmdb_ci kept only as a missing flag	Too sparse to analyse
Missing values (categorical)	u_symptom (23.0%), assignment_group (3.6%), location, category, subcategory (< 0.3%) at logging	isna() on the incident file	Explicit "Missing" category	Keeps all incidents; missingness may be informative
Missing values (time)	resolved_at: 1,556 incidents (6.2%)	isna()	Left missing; resolution time descriptive only	Does not affect the outcome, which is complete
Duplicates	All columns; number + sys_mod_count	duplicated()	0 exact duplicates; 5 key ties broken by sys_updated_at; 0 duplicate incidents	Defines first and final records uniquely
Anomalous code	incident_state = -100 (5 rows)	value_counts()	Retained; final state is Closed	No effect on the outcome
Outliers	reassignment_count, sys_mod_count, resolution_hours	IQR rule (Q3 + 1.5 × IQR)	Retained; banding and log scale	Valid extreme cases, central to the problem
Rare levels	category (22 of 52 levels below 30 incidents); high-cardinality RQ4 fields	Frequency counts	Grouped as Other (regression); one-hot with minimum frequency 30 in training data (RQ4)	Stable estimates; avoids empty cells

Note. Counts are from notebook 02_cleaning_and_eda.ipynb.

Reproducible workflow. Every preprocessing step is scripted rather than done by hand, in this order: (1) notebook 01 loads the raw file, converts the "?" code to missing, parses timestamps, and checks the expected shape (141,712 rows, 36 columns, 24,918 incidents); (2) it sorts events by incident, update count, and update time, keeps the first and final record of each incident, and derives sla_breach; (3) it writes data/processed/incidents.csv; (4) notebook 02 runs the checks in Table 4 and produces Tables 5 and 6 and Figures 3 to 8. Running the four

notebooks in order with the library versions pinned in requirements.txt and random seed 42 regenerates every number in this report (Appendix B).

EDA Results

Table 5 summarises the numeric variables. All are strongly right-skewed: the median incident is never reassigned, is updated four times, and is resolved in 22.1 hours, but the mean resolution time is 178.2 hours because a minority of incidents stay open for weeks. Medians and percentiles are therefore more representative than means, and the counts are not suitable for methods that assume normality.

Table 5

Summary Statistics of Numeric Variables (Incident Level)

Variable	N	Mean	SD	Median	IQR (Q1 – Q3)	95th pct.	Max	Above IQR fence
Reassignment count	24,918	0.94	1.57	0	0 – 1	4	27	2,848
Reopen count	24,918	0.01	0.15	0	0 – 0	0	8	275
Update count	24,918	6.68	8.55	4	2 – 8	19	129	1,470
Resolution hours	23,362	178.17	532.79	22.1	0.42 – 148.48	710.52	8,070.17	2,383

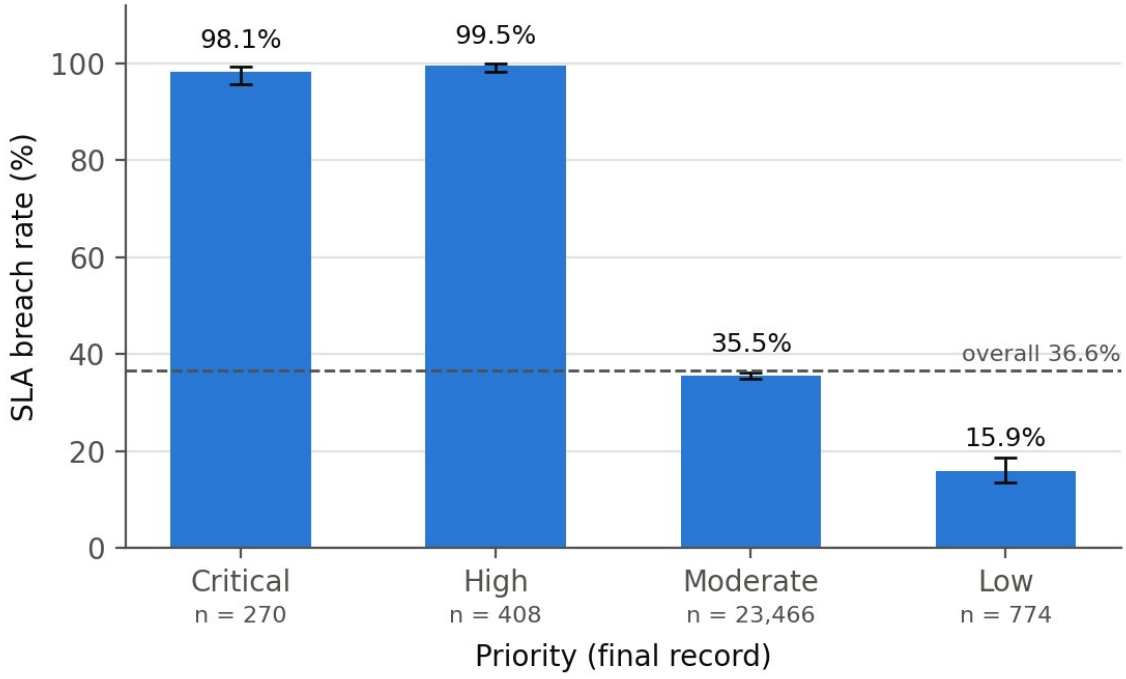
Note. SD = standard deviation; IQR = interquartile range. SLA breach (binary) has a mean of 0.366. Resolution hours exclude 1,556 incidents with no resolution timestamp.

Priority (RQ1). Figure 3 shows a steep breach gradient by priority. Critical and High incidents breach almost always (98.1% and 99.5%), Moderate incidents breach 35.5% of the time, and Low incidents 15.9%. Because Moderate incidents are 94.2% of volume, they account for 8,321 of the 9,115 breaches (91.3%). The near-certain breach of urgent tickets suggests that their targets are very tight relative to how these incidents are worked, while the size of the Moderate group means that most of the business impact, and most of the room for improvement,

sits there. This pattern motivates the formal test in RQ1 and shows why a priority-only rule cannot separate risky from safe Moderate tickets, which is the gap RQ4 addresses.

Figure 3

SLA Breach Rate by Priority

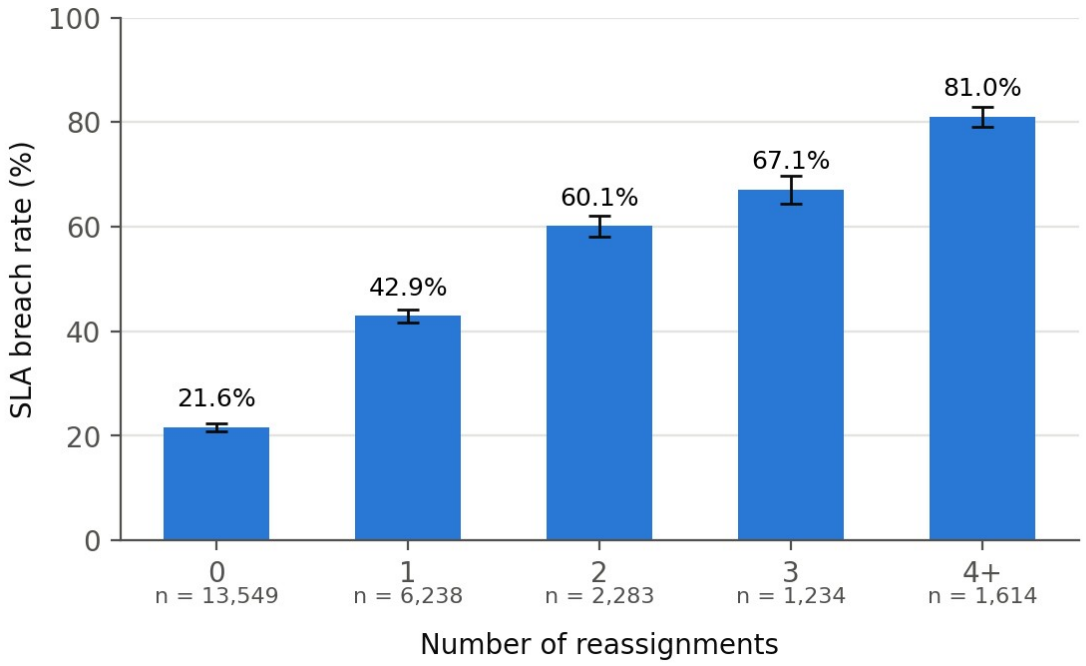


Note. Error bars are Wilson 95% confidence intervals. The dashed line is the overall breach rate.

Reassignment (RQ2). Figure 4 shows that the breach rate rises with every reassignment: 21.6% for incidents never reassigned, 42.9% after one reassignment, 60.1% after two, 67.1% after three, and 81.0% after four or more. Incidents reassigned at least once are 45.6% of all incidents but account for 67.9% of breaches. A monotonic dose–response pattern of this kind is consistent with each hand-off consuming SLA time, but it could also reflect harder incidents being passed around more, which is why RQ2 adjusts for priority and category.

Figure 4

SLA Breach Rate by Number of Reassignments

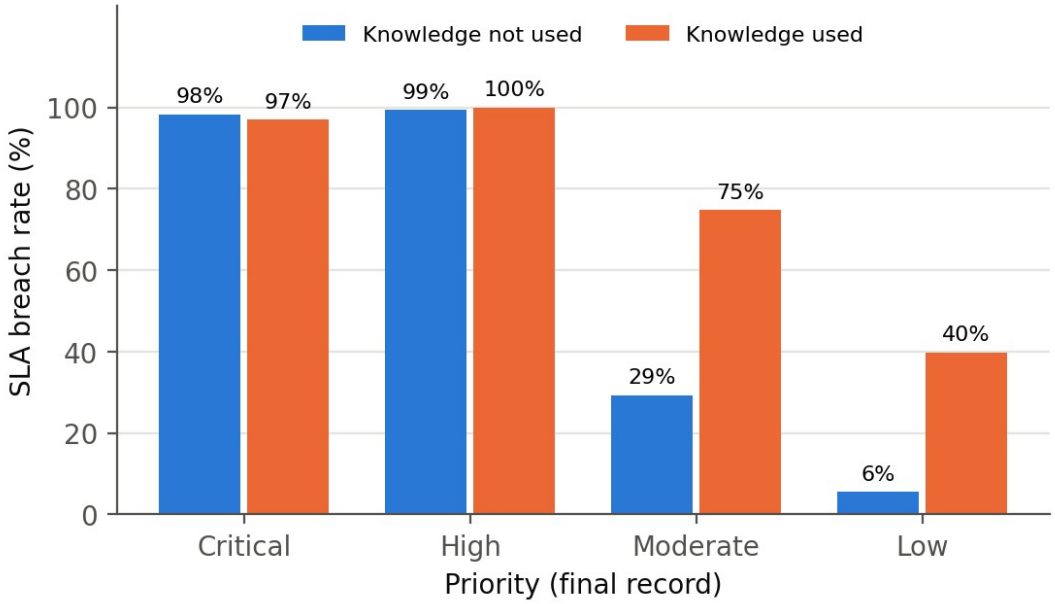


Note. Error bars are Wilson 95% confidence intervals.

Knowledge-base use (RQ3). Figure 5 compares breach rates for incidents with and without knowledge-base use within each priority. For Moderate and Low incidents, those that used a knowledge article breached far more often (75% against 29%, and 40% against 6%), while for Critical and High incidents breach is near-certain either way. A raw comparison would suggest that knowledge articles cause breaches, which is implausible, so the timing of knowledge use was examined next.

Figure 5

SLA Breach Rate by Knowledge-Base Use Within Each Priority



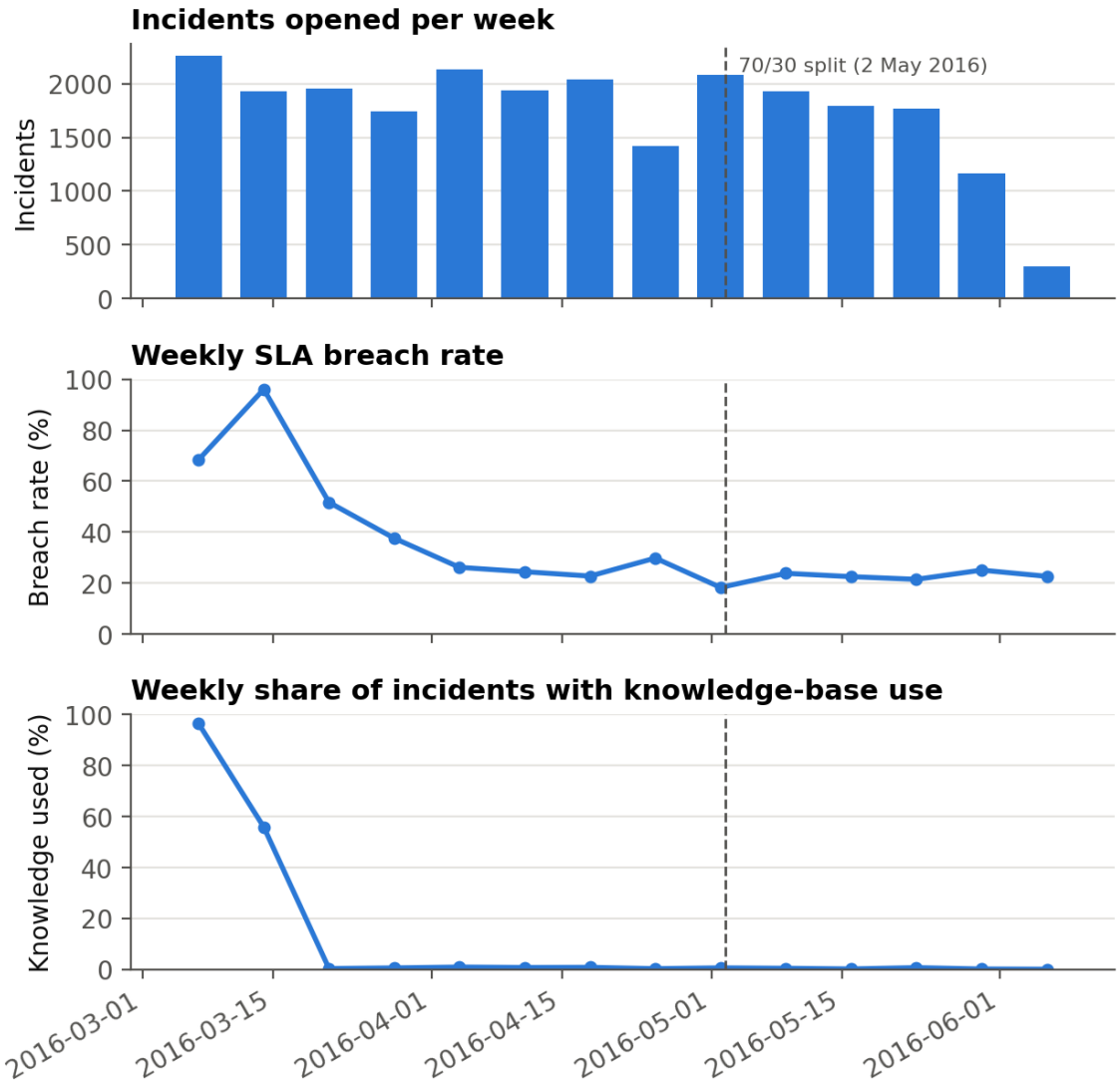
Note. Bars show the share of incidents that breached their SLA, with and without knowledge-base use, within each priority level. Data from Amaral et al. (2018b).

Change over time: a structural break (RQ3 and RQ4). Figure 6 shows weekly volume, breach rate, and knowledge use. Weekly volume is broadly stable, mostly between about 1,700 and 2,260 incidents in full weeks, but the breach rate is 68.3% in the week ending 7 March 2016 and 96.1% in the week ending 14 March, then falls to between 18% and 30% from April onwards. Over the same weeks the knowledge flag drops from 97% of incidents to under 1%. Of the 5,906 incidents opened before 21 March, 58.6% have the knowledge flag set and 73.1% breached; after that date only 0.5% have the flag and 25.2% breached. The most likely explanation is a change in process or recording in mid-March 2016, such as a change in how the knowledge field was filled in, together with an early backlog. The knowledge flag is therefore largely a marker of the period, which confounds the RQ3 comparison, and the breach rate differs

between the RQ4 training period (42.3%) and test period (23.3%). Both points are handled in the preliminary results.

Figure 6

Weekly Incident Volume, SLA Breach Rate, and Knowledge-Base Use



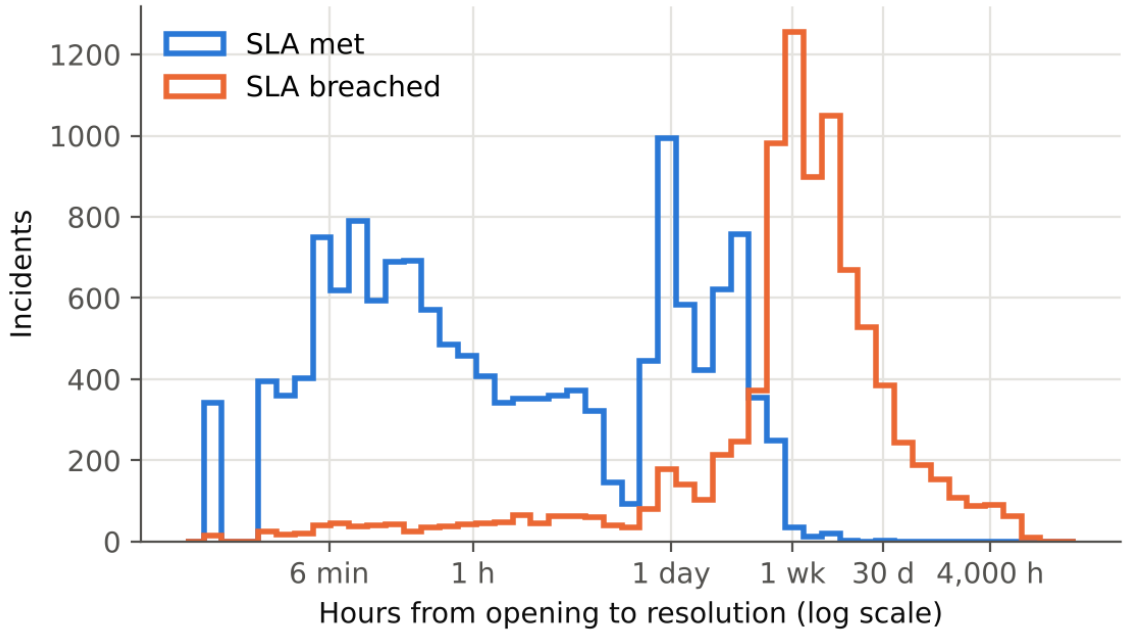
Note. Weeks end on the date shown. Weeks with fewer than 300 incidents (the week ending 29 February 2016 and all weeks after 6 June 2016) are omitted. The dashed line marks the 70/30 temporal split used for RQ4.

Resolution time (outcome check). Figure 7 shows that resolution time is bimodal for incidents that met their SLA, with peaks at minutes and at about one day, while breached

incidents cluster around one to four weeks. The separation confirms that made_sla = true means the SLA was met, and the long right tail (812 incidents over 1,000 hours) explains the skew in Table 5. Resolution time is a consequence of the outcome rather than a predictor available at logging, so it is not used in any model.

Figure 7

Resolution Time by SLA Outcome



Note. Excludes 1,556 incidents with no resolution timestamp and 39 with zero duration. Resolution time is used only to describe the data. Data from Amaral et al. (2018b).

Time of logging (RQ4 features). Figure 8 shows that incidents logged during business hours breach more often (about 38% for 08:00 to 17:59) than those logged in the evening or overnight (about 28%). Separately, weekend incidents breach less often than weekday incidents (24.2% against 37.2%). A plausible reason is queue load: daytime tickets join longer queues. Because the hour and day of logging are known when a ticket is created, they are legitimate predictors for RQ4. Finally, the 52 categories are concentrated (the top 10 hold 75.9% of

incidents) and their breach rates range from 19.4% to 51.6% among the eight largest, which supports controlling for category in RQ2 and RQ3.

Figure 8

SLA Breach Rate by Hour of Logging



Note. Hour of logging is taken from the first event record of each incident. Data from Amaral et al. (2018b).

Table 6 summarises the insight from each figure and table and the decision it leads to.

Table 6

EDA Insight Summary

Figure / table	What it shows	Key insight	Why it matters (RQ link)	Decision / next step
Table 5	Numeric distributions	Strong right skew; long tails are genuine	Model choice (all RQs)	Use medians, banding, non-parametric views
Figure 3	Breach rate by priority	Critical/High breach ~99%; Moderate holds 91% of breaches	RQ1; limits of a priority rule (RQ4)	Test with χ^2; focus the model on Moderate risk
Figure 4	Breach by reassignments	Monotonic rise from 21.6% to 81.0%	RQ2	Adjusted logistic regression; linearity check
Figure 5	Breach by knowledge use	Knowledge users breach more within priority	RQ3	Investigate confounding
Figure 6	Weekly trends	Mid-March 2016 break in breach rate and knowledge flag	RQ3 confounding; RQ4 prevalence shift	Week-adjusted RQ3; stable-period RQ4 check
Figure 7	Resolution time by outcome	Clear separation; confirms made_sla polarity	DV validity (all RQs)	Exclude resolution time from models
Figure 8	Breach by hour of logging	Daytime logging breaches more; weekends breach less (24.2% vs 37.2%)	RQ4 features	Include hour and weekend features

Sample Size Computation

Minimum sample sizes were computed a priori for each research question, matching the method to the planned analysis, with $\alpha = 0.05$, power = 0.80, and Cohen's (1988/2013) small effect size $w = 0.10$ throughout. For the χ^2 tests the required N is λ / w^2, where λ is the noncentrality parameter of the χ^2 distribution that gives 80% power (Faul et al., 2007). RQ2 and RQ3 inflate this by $1 / (1 - R^2)$, where R^2 is the share of the variance in the independent variable explained by the control variables. RQ4 uses the confidence-interval method of Hanley and McNeil (1982) to size the test set so that an AUC of 0.75 is estimated within ±0.03. Table 7 shows the computation; Appendix A gives the code.

Table 7

Sample Size Computation by Research Question

RQ	Method	Parameters	Computation	Minimum N
RQ1	Power: χ^2 test, 4 × 2	$\alpha = 0.05$; power = 0.80; $w = 0.10$; df = 3	$\lambda = 10.903$; $N = 10.903 / 0.10^2 = 1{,}090.3$	1,091
RQ2	Power: χ^2 test, 2 × 2, covariate-adjusted	$\alpha = 0.05$; power = 0.80; $w = 0.10$; df = 1; $R^2 = 0.047$	$\lambda = 7.849$; $785 / (1 - 0.047) = 823.7$	824
RQ3	Power: χ^2 test, 2 × 2, covariate-adjusted	$\alpha = 0.05$; power = 0.80; $w = 0.10$; df = 1; $R^2 = 0.154$	$785 / (1 - 0.154) = 927.9$	928
RQ4	Confidence interval: AUC precision	AUC = 0.75; e = 0.03; z = 1.96; prevalence 0.366; 30% test set	Test set n = 1,191 (436 / 755); 1,191 / 0.30	3,970
Final	**N = max(RQ1–RQ4)**	Available: 24,918 incidents	**max(1,091; 824; 928; 3,970)**	**3,970**

Note. λ = noncentrality parameter; R^2 values are observed in the data; e = half-width of the 95% confidence interval for AUC. Results are rounded up.

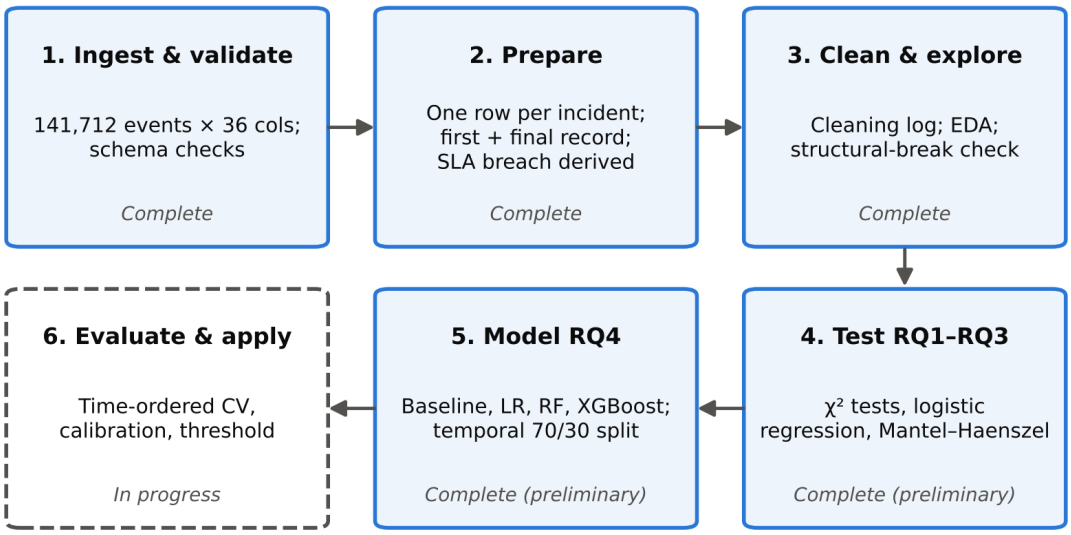
The final sample size is **N = max(1,091; 824; 928; 3,970) = 3,970 incidents**, driven by RQ4. The dataset holds 24,918 incidents, 6.3 times this minimum, so all incidents are used. Two checks confirm logical consistency with the data as analysed. First, the smallest expected cell count in the RQ1 table is 98.8, well above the minimum of 5 for a χ^2 test. Second, the RQ4 test set has 7,476 incidents but a lower breach prevalence (23.3%) than assumed; recomputing with that prevalence requires a test set of 1,681, which the actual test set still exceeds by 4.4 times, and the achieved 95% confidence half-width for the best model's AUC is about ±0.012.

Modelling

The modelling follows the pipeline in Figure 9. Explanatory models answer RQ1 to RQ3, and a set of classifiers answers RQ4. Every step is scripted in the notebooks listed in Appendix B, with a fixed random seed (42), so all results can be reproduced from the repository.

Figure 9

Analysis Pipeline and Interim Status



Note. Solid boxes mark completed stages; the dashed box marks work in progress for the final report.

Choice of Models with Justification

Explanatory models (RQ1 to RQ3). The outcome is binary and the predictors are categorical or counts, so the χ^2 test of independence is the natural first test, reported with Cramér's V or φ as effect size because with 24,918 incidents almost any difference is significant. Pairwise χ^2 tests with Holm (1979) correction identify which priorities differ. Because reassignment and knowledge use are not randomly assigned and vary with priority and category, multivariable logistic regression estimates their adjusted odds ratios, with a Wald test on the coefficient (Hosmer et al., 2013). The Mantel–Haenszel odds ratio (Mantel & Haenszel, 1959) pools the knowledge effect across priority strata as a robustness check, and the Breslow–Day test checks whether that effect is the same in every stratum.

Model 1: priority-only logistic regression (baseline). This reproduces the rule service desks already use, which is to escalate by priority, so beating it is the meaningful benchmark for

RQ4. **Model 2: regularised logistic regression.** It is interpretable through odds ratios, is a strong benchmark for categorical data, and its L1 or L2 penalty, chosen by cross-validation, controls overfitting across 255 encoded columns. **Model 3: random forest.** An ensemble of decision trees (Breiman, 2001) that captures interactions, such as an assignment group that is risky only for certain categories, without manual specification, and is robust to skewed inputs. **Model 4: XGBoost.** Gradient-boosted trees (Chen & Guestrin, 2016), which are often the most accurate models on tabular data (Grinsztajn et al., 2022) and were among the strongest methods in outcome-oriented process benchmarks (Teinemaa et al., 2019).

Assumptions and alternatives. All models assume that incidents are independent, which the collapse to one row per incident supports. The logistic models assume linearity in the logit for counts, which is checked by banding reassignments, and no perfect separation, which holds because even Critical and High incidents contain some non-breaches. The tree ensembles make no linearity or distributional assumptions, but, like all the classifiers, they assume that the training period represents the test period and that coded categories keep the same meaning over time, and boosted trees rely on depth and learning-rate limits to avoid overfitting; the structural break in Figure 6 shows that this assumption is only partly met, which is tested below. Sequence models such as LSTM networks (Tax et al., 2017) were not chosen because at logging there is no event sequence yet, and target encoding of high-cardinality fields (Micci-Barreca, 2001) is kept as a refinement for the final report.

Features Included and Feature Engineering

RQ4 uses 17 features, all taken from the first event record of each incident so that they are known when the ticket is logged (Table 8). Categorical features are one-hot encoded; levels seen fewer than 30 times in the training data are pooled into an "infrequent" level, and missing

values form their own level, which gives 255 encoded columns. The caller is represented by how many incidents the same caller logged in the training period, computed on training data only. Hour of day is encoded with sine and cosine terms so that 23:00 and 00:00 are close, and numeric features are standardised for logistic regression. To avoid leakage (Kaufman et al., 2012), the model excludes everything that accumulates after logging: reassignment, reopen, and update counts, resolution and closure times, the closure code, and final-record priority. Knowledge use is excluded from the main model because Figure 6 shows it acts as a marker of the period, and is tested only in a sensitivity model. Class imbalance is handled with balanced class weights.

Table 8

Feature Set and Usage (RQ4)

Feature	Original / engineered	Type	Reason for inclusion	Used in model(s)
log_priority	Original	Ordinal (one-hot)	Current escalation rule; Figure 3	All (only feature of Model 1)
log_impact, log_urgency	Original	Ordinal (one-hot)	Inputs to priority; finer detail	Models 2–4
log_category, log_subcategory	Original	Nominal (one-hot, pooled)	Service area; breach varies by category	Models 2–4
log_u_symptom	Original	Nominal (one-hot, pooled; Missing level)	User-reported symptom at logging	Models 2–4
log_assignment_group	Original	Nominal (one-hot, pooled; Missing level)	First resolver group; routing quality	Models 2–4
log_location, log_contact_type, log_notify	Original	Nominal (one-hot)	Context of the request	Models 2–4
log_u_priority_confirmation	Original	Binary	Whether priority was checked	Models 2–4
cmdb_ci_missing	Engineered	Binary flag	Configuration item recorded or not	Models 2–4
caller_freq	Engineered	Numeric, log(1 + n)	Frequent callers may differ; avoids 5,244 dummies	Models 2–4

Feature	Original / engineered	Type	Reason for inclusion	Used in model(s)
hour_sin, hour_cos, weekend, day of week	Engineered	Numeric / binary / one-hot	Daytime and weekday effects (Figure 8)	Models 2–4
log_knowledge	Original	Binary	Period marker; tested separately	Sensitivity model only

Note. Excluded to prevent leakage: reassignment_count, reopen_count, sys_mod_count, resolved_at, closed_at, closed_code, and final-record priority.

Evaluation Metrics

Table 9 defines the metrics. AUC is the primary metric because it does not depend on a threshold and is the quantity tested in $H1_4$; precision–recall AUC, recall, and F1 describe performance on the breach class, and the Brier score measures probability accuracy. Validation uses a temporal split: incidents are ordered by opening time, the earliest 70% (17,442 incidents, opened before 11:30 on 2 May 2016) train the models, and the latest 30% (7,476) test them, which mimics deployment on future tickets. Hyperparameters are tuned by five-fold stratified cross-validation within the training set. The worked calculations use the random forest's confusion matrix on the test set at a threshold of 0.5: TP = 1,149, FP = 1,431, FN = 592, TN = 4,304.

Table 9

Evaluation Metrics, Formulae, and Worked Calculations

Metric	Formula	Interpretation	Worked calculation
Accuracy	(TP + TN) / N	Share of all incidents classified correctly; misleading under imbalance	(1,149 + 4,304) / 7,476 = 0.729
Precision	TP / (TP + FP)	Share of flagged incidents that breach	1,149 / 2,580 = 0.445
Recall	TP / (TP + FN)	Share of actual breaches that are flagged	1,149 / 1,741 = 0.660
F1-score	2 × P × R / (P + R)	Balance of precision and recall	2 × 0.445 × 0.660 / 1.105 = 0.532

Metric	Formula	Interpretation	Worked calculation
ROC AUC	P(score of a breach > score of a non-breach)	Ranking quality; 0.5 = chance, 1 = perfect	0.785 (95% CI 0.773–0.796)
PR-AUC	$\Sigma (R_n - R_{n-1}) P_n$ (average precision)	Precision across recall levels; chance = prevalence	0.536 against a chance level of 0.233
Brier score	$(1 / N) \Sigma (p_i - y_i)^2$	Accuracy of predicted probabilities; lower is better	0.181
χ^2, Cramér's V	$\Sigma (O - E)^2 / E$; $\sqrt{(\chi^2 / (N(k - 1)))}$	Association test and its strength (RQ1–RQ3)	RQ1: $\sqrt{(1293.1 / 24{,}918)} = 0.228$
Odds ratio	e^{β}; 95% CI $e^{\beta \pm 1.96\ SE}$	Change in breach odds (RQ2–RQ3)	RQ2: $e^{0.667} = 1.948$

Note. TP, FP, FN, and TN are true positives, false positives, false negatives, and true negatives, with breach as the positive class; P = precision; R = recall; p_i = predicted probability; y_i = observed outcome.

Preliminary Results

This section reports the preliminary results for each research question (notebooks 03 and 04). Table 10 summarises the hypothesis tests for RQ1 to RQ3. P-values below .001 are reported as $p < .001$.

Table 10

Hypothesis Test Results for RQ1 to RQ3

RQ	Test	Result	Effect size	Decision
RQ1	χ^2 test of independence, priority × breach (4 × 2)	$\chi^2(3, N = 24{,}918) = 1{,}293.1$, $p < .001$	$V = 0.228$	Reject $H0_1$
RQ2	χ^2 test, reassigned × breach (2 × 2)	$\chi^2(1) = 2{,}868.4$, $p < .001$; OR = 4.33 [4.1, 4.58]	$\varphi = 0.339$	
	Logistic regression, reassignment count + priority + category (Wald)	OR = 1.948 [1.896, 2.002] per reassignment, $z = 48.46$, $p < .001$	McFadden $R^2 = 0.217$	Reject $H0_2$
RQ3	χ^2 test, knowledge × breach (2 × 2)	$\chi^2(1) = 2{,}399.9$, $p < .001$; OR = 6.24 [5.76, 6.76]	$\varphi = 0.31$	
	Logistic regression, knowledge + priority + category (Wald)	OR = 6.62 [6.05, 7.25], $z = 41.01$, $p < .001$	McFadden $R^2 = 0.171$	
	Mantel–Haenszel across priority strata	OR = 7.26 [6.68, 7.89]; Breslow–Day $p = .006$		
	Logistic regression adding opening week	OR = 0.61 [0.44, 0.86], $p = .004$		Reject $H0_3$ (direction to confirm)

Note. OR = odds ratio with 95% confidence interval in brackets. Category controls use 31 groups (categories with fewer than 30 incidents pooled).

Preliminary Findings by Research Question

RQ1: priority. The breach rate differs strongly by priority, $\chi^2(3, N = 24{,}918) = 1{,}293.1$, $p < .001$, with Cramér's V = 0.228, a small-to-medium effect by Cohen's (1988/2013) benchmarks. Adjusted standardised residuals show more breaches than expected for Critical (+21.1) and High (+26.6) incidents and fewer for Moderate (-14.8) and Low (-12.1). Table 11 shows that five of the six pairwise comparisons remain significant after Holm correction; only Critical and High do not differ (98.1% against 99.5%), because both breach almost always. $H0_1$ is rejected. The finding answers RQ1: priority is clearly associated with breach, but in the opposite direction to its purpose, since the most urgent incidents are the least likely to meet their targets.

Table 11

Pairwise Comparisons of Breach Rates Between Priorities (RQ1)

Comparison	Breach rates	$\chi^2(1)$	Holm-adjusted p	Significant
Critical vs High	98.1% vs 99.5%	2.9	.086	No
Critical vs Moderate	98.1% vs 35.5%	454.3	< .001	Yes
Critical vs Low	98.1% vs 15.9%	580.0	< .001	Yes
High vs Moderate	99.5% vs 35.5%	709.4	< .001	Yes
High vs Low	99.5% vs 15.9%	755.5	< .001	Yes
Moderate vs Low	35.5% vs 15.9%	126.4	< .001	Yes

RQ2: reassignment. Reassigned incidents are more likely to breach, $\chi^2(1) = 2{,}868.4$, $p < .001$, $\varphi = 0.339$. After controlling for priority and category, each additional reassignment multiplies the odds of breach by 1.948 (95% CI 1.896–2.002), Wald z = 48.46, $p < .001$, and the variance inflation factor of 1.049 shows almost no overlap with the controls. The banded model gives odds ratios of 2.88, 5.89, and 12.78 for one, two, and three or more reassignments

compared with none. The first hand-off therefore carries the largest single step, which a single per-reassignment coefficient understates, so the banded form will be reported in the final analysis. Adding the opening week leaves the effect almost unchanged (OR 1.93), so it is robust to the March 2016 break. $H0_2$ is rejected: reassignment is strongly and independently associated with SLA breach.

RQ3: knowledge-base use. The unadjusted association is large, $\chi^2(1) = 2{,}399.9$, $p < .001$, with an odds ratio of 6.24, and it survives adjustment for priority and category (OR 6.62) and Mantel–Haenszel pooling (OR 7.26). The Breslow–Day test ($p = .006$) shows that the effect differs by stratum: it is large for Moderate and Low incidents and absent for Critical and High, where nearly all incidents breach. However, Figure 6 showed that the knowledge flag marks the period before mid-March 2016 rather than individual engineer behaviour. When the opening week is added to the model, the direction reverses: within the same week, knowledge use is associated with lower odds of breach (OR 0.61, 95% CI 0.44–0.86, $p = .004$). $H0_3$ is rejected in every specification, but the preliminary conclusion is that the raw positive association is driven by the period, and that the true direction may be protective. Because fewer than 100 incidents after 21 March used a knowledge article, this estimate is imprecise and will be examined further.

Preliminary Model Performance

Table 12 compares the four RQ4 models on the temporal test set, and Figure 10 shows their ROC curves. The priority-only baseline reaches an AUC of only 0.552, barely above chance, because 95.0% of incidents have the same priority (Moderate) at logging. All three machine-learning models do far better: regularised logistic regression 0.779, random forest 0.785, and XGBoost 0.738. The DeLong test shows that the random forest's AUC is significantly

higher than the baseline's (difference 0.233, z = 38.45, one-sided p < .001), so $H0_4$ is rejected; the logistic regression and XGBoost models are also significantly better than the baseline.

Table 12

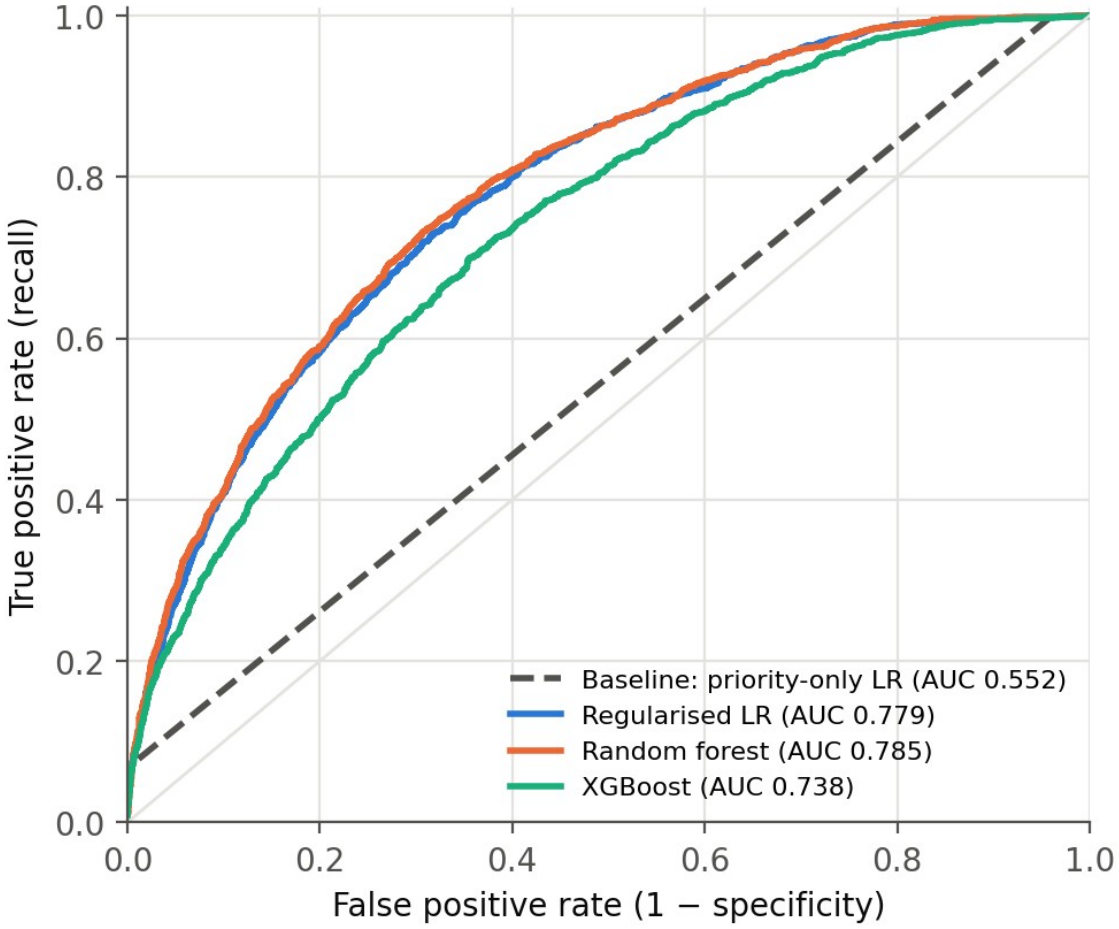
Preliminary Model Performance Comparison (Temporal Test Set)

Model	Feature set	Validation approach	CV AUC	Test AUC [95% CI]	PR-AUC	Precision / recall / F1	Brier	Key takeaway
Model 1: Baseline LR	Priority only	Temporal 70/30	0.533	0.552 [0.545, 0.558]	0.289	0.884 / 0.070 / 0.130	0.240	Priority rule barely beats chance
Model 2: Regularised LR	17 at-logging	Temporal 70/30; 5-fold CV	0.768	0.779 [0.768, 0.791]	0.532	0.436 / 0.655 / 0.523	0.185	Close second; most interpretable
Model 3: Random forest	17 at-logging	Temporal 70/30; 5-fold CV	0.790	0.785 [0.773, 0.796]	0.536	0.445 / 0.660 / 0.532	0.181	Preliminary best; stable from CV to test
Model 4: XGBoost	17 at-logging	Temporal 70/30; 5-fold CV	0.787	0.738 [0.725, 0.751]	0.481	0.400 / 0.605 / 0.481	0.195	Largest CV-to-test drop

Note. Test set: 7,476 incidents (1,741 breaches, prevalence 0.233). CV AUC = mean five-fold cross-validated AUC in the training set. Precision, recall, and F1 use a 0.5 threshold. Best settings: logistic regression C = 1 (l1); random forest min_samples_leaf = 2, max_features = sqrt; XGBoost max_depth = 7, learning rate = 0.1.

Figure 10

ROC Curves of the Four Models on the Temporal Test Set



Note. Test set of 7,476 incidents opened after 11:30 on 2 May 2016. The dashed line is the priority-only baseline; the light diagonal is chance (AUC = 0.5).

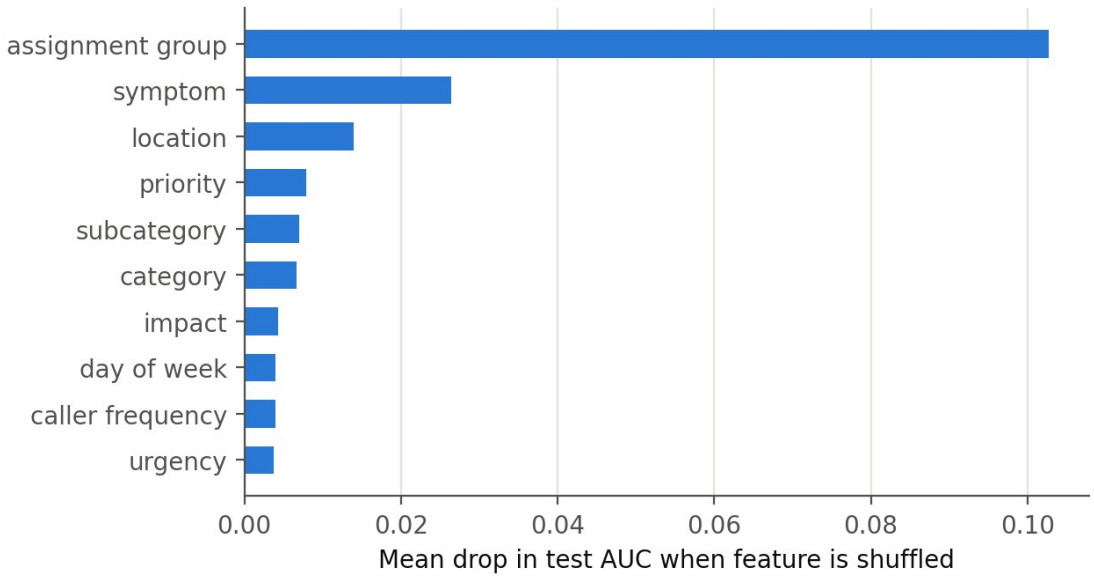
Three further points matter for interpretation. First, accuracy is misleading here: the baseline has the highest accuracy (0.781) only because it flags almost nothing, catching 7.0% of breaches, whereas the random forest catches 66.0% of breaches with a precision of 0.445, nearly double the 0.233 chance level. Its PR-AUC (0.536) is also almost double the baseline's (0.289), and its Brier score is lower (0.181 against 0.240). Second, XGBoost had almost the same cross-validated AUC as the random forest (0.787 against 0.790) but dropped to 0.738 on the test set. Random cross-validation mixes the pre-break and post-break periods, so it rewards patterns that

do not carry forward in time; deeper boosted trees appear to have learned more of them. Third, the random forest is the most stable model across validation and test and is the preliminary best model.

Figure 11 shows the permutation importance of the random forest's features. The first assignment group dominates: shuffling it lowers the test AUC by 0.103, against 0.026 for symptom and 0.014 for location, while priority adds only 0.008. This complements RQ2: where a ticket first lands matters more for its SLA than how urgent it is. Two sensitivity checks support the result. Adding the knowledge flag changes the AUC only from 0.785 to 0.788, so excluding it costs nothing. Refitting on the stable period only (incidents opened from 21 March 2016, again split 70/30 in time) gives an AUC of 0.803 for the random forest against 0.551 for the baseline, so the advantage does not depend on the March break.

Figure 11

Top 10 Features of the Random Forest by Permutation Importance



Note. Importance is the mean drop in test-set AUC over five random shuffles of each feature.

Table 13 maps each research question to its objective and preliminary answer.

Table 13

Preliminary Answers to the Research Questions

RQ	Objective	Preliminary answer	Decision	Remaining work
RQ1	(a) Breach rate by priority	Strong association (V = 0.228); Critical and High breach ~99%, Low 15.9%	Reject $H0_1$	Final wording; business translation
RQ2	(b) Breach odds per reassignment	OR 1.948 per reassignment; first hand-off has the largest step; robust to period	Reject $H0_2$	Report banded model; interaction with priority
RQ3	(c) Breach odds with knowledge use	Positive raw OR (6.24) is driven by the March 2016 period; within-week OR 0.61	Reject $H0_3$; direction tentative	Period-stratified analysis
RQ4	(d) At-logging model vs priority rule	Random forest AUC 0.785 vs 0.552; DeLong $p < .001$	Reject $H0_4$	Time-aware CV, calibration, threshold

Interim Limitations and Risks

- **Data.** All incidents come from one anonymised organisation and mostly from three months in 2016, so results describe this service desk rather than service desks in general. SLA targets are not recorded, so the near-certain breach of Critical and High incidents cannot be checked against their targets. Coded category and group names limit business interpretation. **Mitigation:** findings are framed as evidence from one enterprise, and the method, not the coefficients, is presented as transferable.
- **Structural break.** The change in mid-March 2016 shifts the breach rate from 42.3% in the training period to 23.3% in the test period and turns the knowledge flag into a period marker. **Mitigation:** week-adjusted models for RQ2 and RQ3, a stable-period check for RQ4, and time-aware cross-validation in the final report.
- **Model.** Random-fold cross-validation overestimated XGBoost's out-of-time performance (CV AUC 0.787 against test AUC 0.738); the 0.5 threshold with balanced class weights

is not a business-optimised operating point; and permutation importance can understate correlated features such as category and subcategory. XGBoost results are also sensitive to library versions: because depths 5 and 7 score almost the same in cross-validation, a rerun in Google Colab selected depth 5 and gave a test AUC of 0.751 rather than 0.738, while every other model changed by 0.001 or less and no conclusion changed.

Mitigation: forward-chaining cross-validation, probability calibration, a threshold chosen for triage capacity, and exact library versions pinned in requirements.txt.

- **Assumptions, reliability, and validity.** RQ1 to RQ3 are observational, so the odds ratios are associations, not causal effects; unmeasured factors such as analyst skill may confound them. Reliability is supported by a fixed seed, a scripted pipeline, and the unchanged source file, so every number in this report can be regenerated. Internal validity is supported by leakage controls and a temporal test set; external validity is limited by the single-organisation data.

Next Steps (for Final Report)

- Replace random-fold tuning with forward-chaining time-series cross-validation and retune all RQ4 models.
- Calibrate the best model's probabilities and choose an alert threshold that matches a realistic triage capacity and report breaches caught per 100 tickets flagged.
- Report the banded reassignment model and test a reassignment × priority interaction for RQ2; complete a period-stratified analysis for RQ3.
- Test target encoding for high-cardinality fields and compare it with the current pooling.
- Translate the results into recommendations for service delivery managers and prepare the final report and presentation.

References

Amaral, C. A. L., Fantinato, M., & Peres, S. M. (2018a). Attribute selection with filter and wrapper: An application on incident management process. In M. Ganzha, L. Maciaszek, & M. Paprzycki (Eds.), *Proceedings of the 2018 Federated Conference on Computer Science and Information Systems* (Annals of Computer Science and Information Systems, Vol. 15, pp. 679–682). IEEE. https://doi.org/10.15439/2018F126

Amaral, C., Fantinato, M., & Peres, S. (2018b). *Incident management process enriched event log* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C57S4H

Amaral, C. A. L., Fantinato, M., Reijers, H. A., & Peres, S. M. (2019). Enhancing completion time prediction through attribute selection. In E. Ziemba (Ed.), *Information technology for management: Emerging research and applications* (Lecture Notes in Business Information Processing, Vol. 346, pp. 3–23). Springer. https://doi.org/10.1007/978-3-030-15154-6_1

Botezatu, M. M., Bogojeska, J., Giurgiu, I., Voelzer, H., & Wiesmann, D. (2015). Multi-view incident ticket clustering for optimal ticket dispatching. In *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1711–1720). ACM. https://doi.org/10.1145/2783258.2788607

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review, 78*(1), 1–3. https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

Cohen, J. (2013). *Statistical power analysis for the behavioral sciences* (2nd ed.). Routledge. https://doi.org/10.4324/9780203771587 (Original work published 1988)

Dang, Y., Lin, Q., & Huang, P. (2019). AIOps: Real-world challenges and research innovations. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)* (pp. 4–5). IEEE. https://doi.org/10.1109/ICSE-Companion.2019.00023

DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics, 44*(3), 837–845. https://doi.org/10.2307/2531595

Faul, F., Erdfelder, E., Lang, A.-G., & Buchner, A. (2007). G*Power 3: A flexible statistical power analysis program for the social, behavioral, and biomedical sciences. *Behavior Research Methods, 39*(2), 175–191. https://doi.org/10.3758/BF03193146

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in Neural Information Processing Systems 35* (pp. 507–520). Curran Associates. https://doi.org/10.52202/068431-0037

Hanley, J. A., & McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology, 143*(1), 29–36. https://doi.org/10.1148/radiology.143.1.7063747

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics, 6*(2), 65–70. https://www.jstor.org/stable/4615733

Hosmer, D. W., Jr., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3rd ed.). Wiley. https://doi.org/10.1002/9781118548387

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data, 6*(4), 1–21. https://doi.org/10.1145/2382577.2382579

Kikuchi, S. (2015). Prediction of workloads in incident management based on incident ticket updating history. In *2015 IEEE/ACM 8th International Conference on Utility and Cloud Computing (UCC)* (pp. 333–340). IEEE. https://doi.org/10.1109/UCC.2015.53

Leitner, P., Michlmayr, A., Rosenberg, F., & Dustdar, S. (2010). Monitoring, prediction and prevention of SLA violations in composite services. In *2010 IEEE International Conference on Web Services* (pp. 369–376). IEEE. https://doi.org/10.1109/ICWS.2010.21

Mantel, N., & Haenszel, W. (1959). Statistical aspects of the analysis of data from retrospective studies of disease. *Journal of the National Cancer Institute, 22*(4), 719–748. https://doi.org/10.1093/jnci/22.4.719

Marrone, M., & Kolbe, L. M. (2011). Impact of IT service management frameworks on the IT organization: An empirical study on benefits, challenges, and processes. *Business & Information Systems Engineering, 3*(1), 5–18. https://doi.org/10.1007/s12599-010-0141-5

Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter, 3*(1), 27–32. https://doi.org/10.1145/507533.507538

Polato, M., Sperduti, A., Burattin, A., & de Leoni, M. (2018). Time and activity sequence prediction of business process instances. *Computing, 100*(9), 1005–1031. https://doi.org/10.1007/s00607-018-0593-x

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), Article e0118432. https://doi.org/10.1371/journal.pone.0118432

Shao, Q., Chen, Y., Tao, S., Yan, X., & Anerousis, N. (2008). Efficient ticket routing by resolution sequence mining. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 605–613). ACM. https://doi.org/10.1145/1401890.1401964

Tax, N., Verenich, I., La Rosa, M., & Dumas, M. (2017). Predictive business process monitoring with LSTM neural networks. In E. Dubois & K. Pohl (Eds.), *Advanced information systems engineering* (Lecture Notes in Computer Science, Vol. 10253, pp. 477–492). Springer. https://doi.org/10.1007/978-3-319-59536-8_30

Teinemaa, I., Dumas, M., La Rosa, M., & Maggi, F. M. (2019). Outcome-oriented predictive process monitoring: Review and benchmark. *ACM Transactions on Knowledge Discovery from Data*, *13*(2), 1–57. https://doi.org/10.1145/3301300

van der Aalst, W. (2016). *Process mining: Data science in action* (2nd ed.). Springer. https://doi.org/10.1007/978-3-662-49851-4

Verenich, I., Dumas, M., La Rosa, M., Maggi, F. M., & Teinemaa, I. (2019). Survey and cross-benchmark comparison of remaining time prediction methods in business process monitoring. *ACM Transactions on Intelligent Systems and Technology*, *10*(4), 1–34. https://doi.org/10.1145/3331449

Appendix A

Sample Size Calculation Code

The code below, saved as src/sample_size.py in the GitHub repository, reproduces Table 7.

```
# QM640 capstone - minimum sample sizes (alpha = 0.05, power = 0.80)
# Dependent variable for all RQs: SLA breach (binary)
import numpy as np
from scipy.stats import chi2, ncx2

def chi2_n(w, df, a=0.05, p=0.80):            # chi-square test of independence
    n, c = 10, chi2.ppf(1 - a, df)
    while ncx2.sf(c, df, n * w**2) < p:
        n += 1
    return n

def auc_test_n(auc, e, prev):                 # RQ4 (Hanley & McNeil, 1982)
    q1, q2 = auc / (2 - auc), 2 * auc**2 / (1 + auc)
    n = 50
    while True:
        pos = round(n * prev); neg = n - pos
        var = (auc*(1-auc) + (pos-1)*(q1-auc**2) + (neg-1)*(q2-auc**2)) / (pos*neg)
        if 1.96 * np.sqrt(var) <= e:
            return n
        n += 1

rq1 = chi2_n(0.10, 3)                         # priority (4) x breach (2): 1091
rq2 = int(np.ceil(chi2_n(0.10, 1) / (1 - 0.047)))  # 785 / 0.953 -> 824
rq3 = int(np.ceil(chi2_n(0.10, 1) / (1 - 0.154)))  # 785 / 0.846 -> 928
test = auc_test_n(0.75, 0.03, 9115 / 24918)   # observed prevalence 0.366: 1191
rq4 = int(np.ceil(test / 0.30))               # 3970
print(rq1, rq2, rq3, test, rq4, "final N =", max(rq1, rq2, rq3, rq4))
```

Appendix B

Notebook Map and Run Order

Run the notebooks in order from the notebooks folder after installing requirements.txt.

Each notebook writes its figures to figures/ and its numbers to results/.

Notebook	Purpose	Produces
01_data_preparation.ipynb	Load, validate, collapse events to incidents, derive variables	data/processed/incidents.csv
02_cleaning_and_eda.ipynb	Cleaning checks and exploratory analysis	Tables 4–6; Figures 3–8
03_rq1_rq3_hypothesis_tests.ipynb	χ^2 tests, logistic regressions, Mantel–Haenszel, week adjustment	Tables 10–11
04_rq4_prediction_models.ipynb	Baseline and three classifiers, DeLong test, importance, sensitivity checks	Tables 9 and 12; Figures 10–11
src/sample_size.py	Minimum sample sizes	Table 7